

cTheorems

sahasatvik

<https://github.com/sahasatvik/typst-theorems>

Contents

1. Setup	1
2. Demonstration	2
3. Features	2
3.1. Numbered theorems	2
3.2. References	3
3.3. Chained numbering	3
3.4. Shared numbering	3
3.5. Proofs	3
3.6. Equation tags	4
3.7. Defer, restate, and display	4
3.8. Formatting	5
4. Acknowledgements	5
A. Function documentation	7
A.1. ctheorems	7
A.2. thm-state	13
A.3. thm-counter	19
B. Themes	21
B.1. thm-themes.ams	21
B.2. thm-themes.stripe	22

This package provides functions that help create numbered theorem environments, taking heavy inspiration from the L^AT_EX packages `amsthm`, `thmtools`, `apxproof`.

A *theorem environment* combines content with automatically updated *numbering* information. Theorem environments can

- share the same counter (*Theorems* and *Lemmas* often do so)
- have their counters attached to headings or other environments (*Corollaries* are often numbered based upon the parent *Theorem*)
- be `<label>-ed` and `@reference-d`
- be restated or deferred to appear later in the document.

This package also introduces a few miscellaneous features related to mathematical writing.

- Proof environments, with *QED* symbols.
- Equation tags.
- Predefined sets of commonly used theorem environments, and a few themes.

1. Setup

```
#import "@preview/ctheorems:2.0.0": *
#show: thm-rules // Must include!
```

Applying the `thm-rules` show rule — in the same `.typ` file where `ctheorems` functions have been used — is **essential** for displaying theorem environments, references, and equations correctly!

A standard set of theorem environments in the AMS style is available using

```
#import thm-themes.ams: *
```

This lets you immediately use `theorem`, `proof`, `proposition`, `lemma`, `corollary`, `definition`, `example`, `remark`, `claim`. See Appendix B.1 for more details.

2. Demonstration

```
#import "@preview/ctheorems:2.0.0": *
#import thm-themes.ams: *
#show: thm-rules.with(
  qed-symbol: $square$
)

#definition[Expectation][
  The expectation of a random variable $X$ on
  a probability space $(\Omega, \mathcal{E}, \mathbb{P})$ is
  $
  \mathbb{E}[X] = \int X \, d\mathbb{P},
  $
  whenever well-defined.
] <expectation>

#remark[
  We require at least one of $\mathbb{E}[X^+]$,
  $\mathbb{E}[X^-]$ to be finite.
]

#proposition[
  For any $A$ in $\mathcal{E}$, $
  \mathbb{P}(X \in A) = \mathbb{E}[\mathbf{1}_A(X)].
  $
] <prob-exp>

#theorem[Markov][
  Let $X \ge 0$. For all $a > 0$, $
  \mathbb{P}(X > a) \le \mathbb{E}[X] / a.
  $
] <markov>

#corollary[Chebyshev][
  Let $\mathbb{E}[X] = \mu$, "$\text{var}[X] = \sigma^2$".
  Then, $
  \mathbb{P}(|X - \mu| \ge k \sigma) \le 1 / k^2.
  $
] <chebyshev>

#proof[of @markov][
  $
  \mathbb{P}(X > a) \le
  \mathbb{E}[\mathbf{1}_{(a, \infty)}(X)]
  \tag*{tag[[@prob-exp[Prop.]]]}
  \le \mathbb{E}[(X / a) \mathbf{1}_{(a, \infty)}(X)] \le
  \mathbb{E}[X] / a. \quad \text{\#qedhere}
  $
]
```

Definition 2.1 (Expectation). The expectation of a random variable X on a probability space $(\Omega, \mathcal{E}, \mathbb{P})$ is

$$\mathbb{E}[X] = \int X \, d\mathbb{P},$$

whenever well-defined.

Remark. We require at least one of $\mathbb{E}[X^+]$, $\mathbb{E}[X^-]$ to be finite.

Proposition 2.2. For any $A \in \mathcal{E}$,

$$\mathbb{P}(X \in A) = \mathbb{E}[\mathbf{1}_A(X)].$$

Theorem 2.3 (Markov). Let $X \geq 0$. For all $a > 0$,

$$\mathbb{P}(X > a) \leq \frac{\mathbb{E}[X]}{a}.$$

Corollary 2.3.1 (Chebyshev). Let $\mathbb{E}[X] = \mu$, $\text{var}[X] = \sigma^2$. Then,

$$\mathbb{P}(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}.$$

Proof of Theorem 2.3.

$$\begin{aligned} \mathbb{P}(X > a) &= \mathbb{E}[\mathbf{1}_{(a, \infty)}(X)] \quad (\text{Prop. 2.2}) \\ &\leq \mathbb{E}\left[\left(\frac{X}{a}\right) \mathbf{1}_{(a, \infty)}(X)\right] \\ &\leq \frac{\mathbb{E}[X]}{a}. \quad \square \end{aligned}$$

3. Features

3.1. Numbered theorems

```
#let theorem = thm.with(
  supplement: "Theorem",
  base-level: 1
)

#theorem("Euclid")[
  There are infinitely many primes.
] <euclid>
```

Theorem 3.1 (Euclid). There are infinitely many primes.

Note that `theorem` inherits its numbering from the current heading (the default `thm.base`). By setting `thm.base-level` to `1`, this theorem only uses the first level count from its base.

3.2. References

Since our theorem has been labeled `<euclid>`, we can reference it elsewhere in the document via `@euclid`.

We will supply a proof of `@euclid` later.

We will supply a proof of Theorem 3.1 later.

3.3. Chained numbering

Theorem environments can be attached together by setting the child's `thm.base` to the parent's `thm.counter`.

```
#let corollary = thm.with(
  supplement: "Corollary",
  base: "Theorem"
)

#corollary(rewrite: true)[
  There is no largest prime number.
] <largest-prime>

#proof([of @largest-prime], defer: true)[
  The existence of a largest prime would
  imply the finitude of the set of primes.
]
```

Corollary 3.1.1. *There is no largest prime number.*

We have set `thm.restate` and `thm.defer` — go to Section 3.7 to find out more!

3.4. Shared numbering

Theorem environments can share the same numbering by setting a common `thm.counter`.

```
#let proposition = thm.with(
  supplement: "Proposition",
  counter: "Theorem",
  base-level: 1
)

#proposition[
  If a natural number divides  $a$  and  $b$ , it
  also divides  $a - b$ .
]
```

Proposition 3.2. *If a natural number divides a and b , it also divides $a - b$.*

For directly manipulating theorem counters, see `thm-counter-get`, `thm-counter-step`, `thm-counter-update`.

3.5. Proofs

Theorems go hand in hand with proofs, which are available using `proof`.

```
#proof[of @euclid][
  Suppose to the contrary that $p_1, p_2,
  dots, p_n$ is a finite enumeration of
  all primes. Set $P = p_1 p_2 \dots p_n$.
  Since $P + 1$ is not in our list, it cannot
  be prime. Thus, some prime factor $p_j$
  divides $P + 1$. Since $p_j$ also divides
  $P$, it must divide the difference $(P + 1)
  - P = 1$, a contradiction.
]
```

Proof of Theorem 3.1. Suppose to the contrary that $p_1, p_2, \dots, p_n$ is a finite enumeration of all primes. Set $P = p_1 p_2 \dots p_n$. Since $P + 1$ is not in our list, it cannot be prime. Thus, some prime factor p_j divides $P + 1$. Since p_j also divides P , it must divide the difference $(P + 1) - P = 1$, a contradiction. ■

Proof environments will float a QED symbol (■) at the bottom right by default. The symbol can be customized by setting `thm-rules.qed-symbol`. If your proof ends in a block equation, or a list/enum, you can place `qedhere` to correctly position the QED symbol.

```
#show: thm-rules.with(qed-symbol: $square$)

#theorem[Prime gaps][
  There are arbitrarily long stretches of
  composite numbers.
]
#proof[
  For any $n > 2$, consider $
    n! + 2, quad
    n! + 3, quad \dots, quad
    n! + n. #qedhere
$
]
```

Theorem 3.3 (Prime gaps). *There are arbitrarily long stretches of composite numbers.*

Proof. For any $n > 2$, consider

$$n! + 2, \quad n! + 3, \quad \dots, \quad n! + n. \quad \square$$

Caution: `qedhere` does not play well with numbered equations!

3.6. Equation tags

Tags can be inserted into equations using `tag`. These float to the right, mimicking `\tag` in $\text{\LaTeX}$.

```
$
(a + b)^2
&= a^2 + 2 a b + b^2 \
&\leq 2a^2 + 2b^2 #tag[(AM-GM)] \
&\leq 2 \thickmax\{a, b\}^2.
$
```

$$\begin{aligned} (a + b)^2 &= a^2 + 2ab + b^2 \\ &\leq 2a^2 + 2b^2 \quad (\text{AM-GM}) \\ &\leq 2 \max\{a, b\}^2. \end{aligned}$$

3.7. Defer, restate, and display

Theorem environments can be restated, or deferred to later in the document by setting `thm.restate`, `thm.defer` as in the example in Section 3.3, then calling `thm-restate`.

```
#import thm-state: thm-restate, thm-display

#thm-restate()
```

Corollary 3.1.1. *There is no largest prime number.*

Proof of Corollary 3.1.1. The existence of a largest prime would imply the finitude of the set of primes. ■

This is often useful for pushing content to the appendix. For finer control, consider using `thm.restate-keys`.

```
Euclid's Theorem states:
#thm-restate(
  "Euclid",
  all: true,
  fmt: thm  $\Rightarrow$  thm.body
)
```

Euclid's Theorem states: There are infinitely many primes.

The `thm-display` function is the most powerful method for manipulating theorem environments. The following produces a crude outline of named theorem environments (so far), excluding proofs; see `thm.fmt` for more information about the formatting function `fmt`.

```
#thm-display(
  thm  $\Rightarrow$  thm.name  $\neq$  none and thm.supplement  $\neq$  "Proof", // Filter
  fmt: thm  $\Rightarrow$  { // Format
    let head = [*#thm.supplement~#thm.number*]
    if thm.name  $\neq$  none { head = head + [~(#thm.name)] }
    let page = link(thm.loc, [#thm.loc.position().page]) // Get page number
    [#head~#box(width: 1fr, repeat[.]~#page\ ]
  }
)
```

Definition 2.1 (Expectation)	2
Theorem 2.3 (Markov)	2
Corollary 2.3.1 (Chebyshev)	2
Theorem 3.1 (Euclid)	2
Theorem 3.3 (Prime gaps)	4

Setting `thm-display.final` to `true` would extend this search to the entire document.

3.8. Formatting

The default theorem formatting function, `thm-fmt-block` helps with basic customization of the theorem title and body, along with the surrounding block.

```
#let theorem-standout = theorem.with(
  stroke: 1pt + green,
  fill: green.lighten(95%),
  outset: 0.7em,
  spacing: 1.5em
)

#theorem-standout(
  title-fmt: x  $\Rightarrow$  smallcaps(strong(x))
)[
  There are infinitely many composite
  numbers.
]
```

THEOREM 3.4. *There are infinitely many composite numbers.*

See `thm.fmt` for more information on how to write your own custom formatting function.

4. Acknowledgements

Thanks to

- [MJHutchinson](#) for suggesting and implementing the `base-level` and `base: none` features,

- [rmolinari](#) for suggesting and implementing the `separator: ...` feature,
- [DVDTSB](#) for contributing
 - the idea of passing named arguments from the theorem directly to the `fmt` function.
 - the `number: ...` override feature.
 - the `title: ...` override feature in `thm-plain`.
- [PgBiel](#) for fixing breaking changes in version updates.
- The authors of the $\text{\LaTeX}$ packages `amsthm`, `thmtools`, `apxproof`.
- The awesome devs of [typst.app](#) for their support.

A. Function documentation

A.1. ctheorems

- `thm()`
- `proof()`
- `tag()`
- `thm-rules()`
- `thm-fmt-block()`
- `proof-body-fmt()`

Variables:

- `qedhere`

`thm`

Numbered theorem environment.

```
#thm(base: none)[Name][
  #lorem(10)
]

#thm(supplement: "Corollary", base: "Theorem")[
  #lorem(7)
]
```

Theorem 1 (Name). *Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.*

Corollary 1.1. *Lorem ipsum dolor sit amet, consectetur adipiscing.*

Parameters

```
thm(
  ..args: arguments ,
  body: content ,
  supplement: str content ,
  counter: str auto ,
  base: str none ,
  base-level: int none ,
  name: content ,
  number: content auto none ,
  numbering: str function none ,
  restate: bool ,
  defer: bool ,
  restate-keys: array ,
  fmt: function ,
  ref-fmt: function
) -> figure context
```

..args arguments

Extra arguments. See `thm.name`, `thm.fmt`.

body content

Theorem body.

supplement str or content

Theorem supplement.

Default: "Theorem"

counter `str` or `auto`

Environment counter name. If `auto`, defaults to `thm.supplement`.

```
#thm(  
  supplement: "Proposition",  
  counter: "Theorem",  
  base: none  
) [  
  #lore(7)  
]
```

Proposition 2. *Lorem ipsum dolor sit amet, consectetur adipiscing.*

Default: `auto`

base `str` or `none`

Base counter name, whose numbering prefixes the theorem environment numbering. If `"heading"`, use the heading counter. If `none`, the theorem environment maintains a global count with no prefix.

```
#thm(supplement: "Example", base: "Theorem") [  
  #lore(4)  
]
```

Example 2.1. *Lorem ipsum dolor sit.*

Default: `"heading"`

base-level `int` or `none`

Base level, determining the number of levels of the `base` numbering to use for the theorem environment numbering. If `none`, all levels from the `base` numbering are used. Setting a base level higher than what the `base` provides will introduce zeros for padding.

```
#thm(  
  supplement: "Sub-Sub-Theorem",  
  base: "Theorem",  
  base-level: 2  
) [  
  #lore(12)  
]
```

Sub-Sub-Theorem 2.0.1. *Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor.*

Default: `none`

name `content`

Theorem name. If `auto`, the first positional argument from `thm.args` (if present) is chosen as the theorem environment's name.

```
#thm(name: "Named", base: none) [  
  #lore(4)  
]  
#thm(base: none)[Also Named] [  
  #lore(8)  
]
```

Theorem 3 (Named). *Lorem ipsum dolor sit.*

Theorem 4 (Also Named). *Lorem ipsum dolor sit amet, consectetur adipiscing elit.*

Default: `auto`

number `content` or `auto` or `none`

Theorem number. If `auto` (and `thm.numbering` is not `none`), the number is computed based on `thm.counter`, `thm.base`, `thm.base-level`.

```
#thm(number: $dagger dagger$, base: none)[
  #lorem(5)
] <d-dag>
```

Recall @d-dag.

Theorem ††. *Lorem ipsum dolor sit amet.*

Recall Theorem ††.

Default: `auto`

numbering `str` or `function` or `none`

Theorem numbering style. If `none`, numbering is suppressed.

```
#thm(supplement: "Remark", numbering: none)[
  #lorem(5)
]
```

Remark. *Lorem ipsum dolor sit amet.*

Default: `"1.1"`

restate `bool`

Mark for being restated later. See `thm-restate()`.

Default: `false`

defer `bool`

Mark for being deferred to later, without appearing in the current position. See `thm-restate()`.

Default: `false`

restate-keys `array`

Keys used by `thm-restate.keys` for filtering. If `auto`, defaults to an array containing `thm.supplement` and `thm.name` (if present).

Default: `auto`

fmt `function`

Formatting function controlling the appearance of the theorem environment, of the form `thm ⇒ content`. `thm` is a dictionary whose keys are essentially the arguments of `thm()`, with a couple of additions/exceptions.

- The keys `name`, `counter`, `number`, `restate-keys` contain their finally inferred/computed values.
- The key `loc` contains the theorem's location.
- The key `args` contains a dictionary of only the named arguments from `thm.args`.

It is often useful to use `thm.args` to control formatting functions.

```
#let thm-color = thm.with(
  base: none,
  fmt: thm => {
    let color = thm.args.at("color", default:
black)
    [*#thm.supplement~#thm.number.* #text(color,
thm.body)]
  }
)

#thm-color[#lorem(15)]
#thm-color(color: blue)[#lorem(7)]
```

Default: thm-fmt-block

Theorem 5. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Theorem 6. Lorem ipsum dolor sit amet, consectetur adipiscing.

ref-fmt function

Formatting function for references, with the same form as `thm.fmt`. The `thm` dictionary carries an additional `ref-supplement` key containing the supplement used in the `@cite[ref-supplement]` call.

```
#let thm-ref-name = thm.with(
  base: none,
  ref-fmt: thm => {
    [#thm.name~(#thm.supplement~#link(thm.loc,
thm.number))]
  }
)

#thm-ref-name[Gauss][#lorem(17)] <gauss>

By @gauss, ...
```

Theorem 7 (Gauss). Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore.

By Gauss (Theorem 7), ...

Default: thm => {

```
  let supplement = thm.supplement
  if thm.ref-supplement != none { supplement = thm.ref-supplement }
  if thm.ref-supplement == [!] and thm.name != none {
    [#link(thm.loc, thm.name)]
    return
  }
  if supplement != none and supplement != [] { supplement = [#supplement~] }
  [#supplement#link(thm.loc, thm.number)]
}
```

proof

Proof environment. Identical to `thm()`, with different defaults. Uses the `thm-fmt-block()` formatting function (for `thm.fmt`), with `proof-body-fmt()` (for `thm-fmt-block.body-fmt`).

```
#thm(base: none)[
  #lorem(6)
]
#proof[
  #lorem(3)
]
```

Theorem 8. Lorem ipsum dolor sit amet, consectetur.

Proof. Lorem ipsum dolor. ■

Parameters

`proof()` -> function

tag

Add content which floats to the right in an equation.

```
$
(a + b)^2
&= a^2 + 2 a b + b^2 \
&\leq 2a^2 + 2b^2 #tag[(AM-GM)] \
&\leq 2 \thin \max{a, b}^2.
$
```

$$\begin{aligned}(a + b)^2 &= a^2 + 2ab + b^2 \\ &\leq 2a^2 + 2b^2 \quad (\text{AM-GM}) \\ &\leq 2 \max\{a, b\}^2.\end{aligned}$$

Parameters

```
tag(
  t: any ,
  move: bool
) -> content
```

t any

Content to use as tag

move bool

If **false**, tags occupy no horizontal space in the equation layout. Set this to **true** if equation content overlaps the tag. This will have the minor disadvantage of moving the equation off-center to accommodate the tag.

Note: Trying to determine this option automatically currently causes layout convergence issues in some cases.

Default: **false**

thm-rules

Rules for styling theorem environments, references, proofs, etc. *Must appear at the beginning of the document.*

Parameters

```
thm-rules(
  qed-symbol: content ,
  doc
) -> content
```

qed-symbol content

Symbol displayed at the end of proofs. See [proof\(\)](#), [qedhere](#), [proof-body-fmt\(\)](#). Use as

```
#show: thm-rules.with(
  qed-symbol: $square$
)

#proof[
  #lorem(5)
  $ integral_0^oo sin(x)/x \thin d x = pi/2.
  #qedhere $
]
```

Proof. Lorem ipsum dolor sit amet.

$$\int_0^\infty \frac{\sin(x)}{x} dx = \frac{\pi}{2}. \quad \square$$

Default: **\$qed\$**

thm-fmt-block

Default formatting function for theorem environments, wrapping everything in a block. Intended for use in `thm.fmt`. Named arguments from `thm.args` can override all of the arguments below; any remaining named arguments are passed to the block. Blocks have `width: 100%` set by default.

```
#thm(  
  base: none,  
  fmt: thm-fmt-block,  
  title-fmt: x => strong(smallcaps(x)),  
  separator: [\ ],  
  stroke: (left: 1pt),  
  inset: (x: 0.7em, y: 0.2em)  
)[Special][  
  #lorem(16)  
]
```

THEOREM 9 (Special)

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et.

Parameters

```
thm-fmt-block(  
  thm: dictionary ,  
  block-args: dictionary ,  
  name-fmt: function ,  
  title-fmt: function ,  
  body-fmt: function ,  
  separator: content  
) -> content
```

thm dictionary

Theorem data. See `thm.fmt` for details.

block-args dictionary

Named arguments for the block.

Default: `(:)`

name-fmt function

Formatting for the environment name.

Default: `x => [(#x)]`

title-fmt function

Formatting for the environment title (head and number).

Default: `strong`

body-fmt function

Formatting for the environment body.

Default: `emph`

separator content

Separator between title and body.

Default: [`*.*`]

proof-body-fmt

Used as `thm-fmt-block.body-fmt` by `proof()`, for properly styling proofs with a `qed` symbol inserted at the end of the body. Also see [qedhere](#).

```
#show: thm-rules.with(qed-symbol: $"QED"$)

#proof-body-fmt[#lorem(3)]

#proof-body-fmt[
$
  phi.alt(x) = 1/sqrt(2 pi) e^(-x^2/2) #qedhere
$
]
```

Lorem ipsum dolor. QED

$$\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2} \quad \text{QED}$$

Parameters

`proof-body-fmt` (`body`: `content`) -> `content`

body `content`

Proof body.

qedhere metadata

If placed in a block equation/enum/list within a proof, place a `qed` symbol to its right.

```
#proof[
  #lorem(3)
  $ x^2 + y^2 = z^2. #qedhere $
]

#proof[
+ #lorem(4)
+ #lorem(5) #qedhere
]

#proof[
$
  (a + b)^2 &= (a + b)(a + b) \
    &= a^2 + 2 a b + b^2. #qedhere
$
]
```

Proof. Lorem ipsum dolor.

$$x^2 + y^2 = z^2. \quad \blacksquare$$

Proof.

1. Lorem ipsum dolor sit.
2. Lorem ipsum dolor sit amet.

Proof.

$$\begin{aligned} (a + b)^2 &= (a + b)(a + b) \\ &= a^2 + 2ab + b^2. \end{aligned} \quad \blacksquare$$

A.2. thm-state

Methods for restating, reformatting, filtering, and manipulating theorem environments from elsewhere in the document. Every call to `thm` appends a `thm` data dictionary to the state `thm-stored`, where the keys of `thm` are as described in `thm.fmt`.

Import all functions using

```
#import thm-state: *
```

- `thm-restate()`
- `thm-display()`

Variables:

- `thm-stored`

`thm-restate`

Displays theorem environments which have been marked to be restated or deferred (by setting `thm.restate`, `thm.defer`), can be filtered. Useful for pushing content to the appendix.

```
#let theorem = thm.with(supplement: "Theorem")
#let lemma = thm.with(
  supplement: "Lemma",
  counter: "Theorem",
)
#let definition = thm.with(
  supplement: "Definition",
  body-fmt: x => x
)

= Heading

#definition[#lorem(2)]
#lemma[#lorem(8)]

#theorem("Name", restate: true)[#lorem(7)]
#proof(defer: true)[
  #lorem(7)
]

#lemma[#lorem(4)]

= Appendix

#thm-restate()
```

1 Heading

Definition 1.1. Lorem ipsum.

Lemma 1.1. Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Theorem 1.2 (Name). Lorem ipsum dolor sit amet, consectetur adipiscing.

Lemma 1.3. Lorem ipsum dolor sit.

2 Appendix

Theorem 1.2 (Name). Lorem ipsum dolor sit amet, consectetur adipiscing.

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing. ■

Parameters

```
thm-restate(
  ..keys: str content array function ,
  fmt: function auto ,
  at: label selector location function auto ,
  final: bool ,
  all: bool
) -> content
```

..keys `str` or `content` or `array` or `function`

String/content keys, array of keys, or functions used to filter theorem environments. A theorem environment is displayed if it passes *any* of the filters.

If `k` in `keys` is a `string/content`, theorem environments containing `k` in its array of `restate-keys` will be matched.

```
#let theorem = thm.with(supplement: "Theorem")
#let lemma = thm.with(
  supplement: "Lemma",
  counter: "Theorem",
)
```

= Heading

```
#theorem(restate: true)[#lorem(6)]
#lemma(restate: true)[#lorem(4)]
#proof(defer: true)[#lorem(7)]
#lemma(restate: true)[#lorem(3)]
```

= Restate lemmas/proofs

```
#thm-restate("Lemma", "Proof")
```

1 Heading

Theorem 1.1. *Lorem ipsum dolor sit amet, consectetur.*

Lemma 1.2. *Lorem ipsum dolor sit.*

Lemma 1.3. *Lorem ipsum dolor.*

2 Restate lemmas/proofs

Lemma 1.2. *Lorem ipsum dolor sit.*

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing. ■

Lemma 1.3. *Lorem ipsum dolor.*

If `k` in `keys` is an array of strings/content, theorem environments containing *all* keys from `k` in its array of `restate-keys` will be matched.

= Heading

```
#theorem(
  "Result A",
  restate: true,
  restate-keys: ("Theorem", "Result A")
)[#lorem(6)]
#proof(
  defer: true,
  restate-keys: ("Proof", "Result A")
)[#lorem(7)]
#theorem(restate: true)[#lorem(6)]
#theorem(
  "Result B",
  restate: true,
  restate-keys: ("Theorem", "Result B")
)[#lorem(6)]
#proof(
  defer: true,
  restate-keys: ("Proof", "Result B")
)[#lorem(7)]
```

= Restate Result A

```
#thm-restate("Result A")
```

= Restate theorems tagged Result B

```
#thm-restate(("Theorem", "Result B"))
```

3 Heading

Theorem 3.1 (Result A). *Lorem ipsum dolor sit amet, consectetur.*

Theorem 3.2. *Lorem ipsum dolor sit amet, consectetur.*

Theorem 3.3 (Result B). *Lorem ipsum dolor sit amet, consectetur.*

4 Restate Result A

Theorem 3.1 (Result A). *Lorem ipsum dolor sit amet, consectetur.*

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing. ■

5 Restate theorems tagged Result B

Theorem 3.3 (Result B). *Lorem ipsum dolor sit amet, consectetur.*

If `k` in `keys` is a function, it must be of the form `restate-keys → bool`.

= Heading

```
#theorem(  
  restate: true,  
  restate-keys: (  
    "Theorem", "Unproven claim"  
  )  
)[#lore(6)]  
#theorem(restate: true)[#lore(6)]  
#lemma(  
  "Claim D",  
  restate: true,  
  restate-keys: ("Lemma", "Claim D")  
)[#lore(6)]
```

= Restate claims

```
#thm-restate(  
  keys => keys.any(  
    k => lower(k).contains("claim")  
  )  
)
```

6 Heading

Theorem 6.1. *Lorem ipsum dolor sit amet, consectetur.*

Theorem 6.2. *Lorem ipsum dolor sit amet, consectetur.*

Lemma 6.3 (Claim D). *Lorem ipsum dolor sit amet, consectetur.*

7 Restate claims

Theorem 6.1. *Lorem ipsum dolor sit amet, consectetur.*

Lemma 6.3 (Claim D). *Lorem ipsum dolor sit amet, consectetur.*

fmt **function** or **auto**

Formatting function, with the same form as `thm.fmt`. The default **auto** uses the originally supplied formatting function `thm.fmt`. See `thm-display.fmt`.

Default: **auto**

at **label** or **selector** or **location** or **function** or **auto**

Location up to which theorem environments will be displayed. The default **auto** uses the location where `thm-restate()` was called. See `thm-display.at`.

Default: **auto**

final **bool**

If **true**, display environments up to the end of the document. Overrides `thm-restate.at`. See `thm-display.final`.

Default: **false**

all **bool**

If **true**, ignores the `thm`'s `restate` or `defer` state (see `thm.restate`, `thm.defer`). Calling `thm-restate(all: true)` is equivalent to `thm-display()`.

Default: **false**

thm-display

Displays all theorem environments, can be filtered.

Every call to `thm()` appends a `thm` dictionary to `thm-stored`. A `thm` dictionary contains complete information about a theorem environment; its keys are as described in `thm.fmt`. This lets theorems be restated, filtered, and otherwise manipulated elsewhere in a document.

```
#let theorem = thm.with(supplement: "Theorem")
#let lemma = thm.with(
  supplement: "Lemma",
  counter: "Theorem",
)
#let definition = thm.with(
  supplement: "Definition",
  body-fmt: x => x
)

= Heading <h1>

#theorem("Name")[#lorem(7)]
#proof[
  #lorem(7)
]

= New_heading <h2>

#lemma[#lorem(8)]
#definition("Thing")[#lorem(2)]
#lemma[#lorem(4)]

= Display_all

#thm-display()
```

1 Heading

Theorem 1.1 (Name). *Lorem ipsum dolor sit amet, consectetur adipiscing.*

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing. ■

2 New heading

Lemma 2.1. *Lorem ipsum dolor sit amet, consectetur adipiscing elit.*

Definition 2.1 (Thing). Lorem ipsum.

Lemma 2.2. *Lorem ipsum dolor sit.*

3 Display all

Theorem 1.1 (Name). *Lorem ipsum dolor sit amet, consectetur adipiscing.*

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing. ■

Lemma 2.1. *Lorem ipsum dolor sit amet, consectetur adipiscing elit.*

Definition 2.1 (Thing). Lorem ipsum.

Lemma 2.2. *Lorem ipsum dolor sit.*

Parameters

```
thm-display(
  ..filters: function,
  fmt: function auto,
  at: label selector location function auto,
  final: bool
) -> content
```

..filters `function`

Filtering functions. Each `f` in `filters` is a function `thm => bool`. A theorem environment is displayed if it passes *any* of the filters.

= Display only theorems/proofs

```
#thm-display(  
  thm => thm.supplement == "Theorem",  
  thm => thm.supplement == "Proof",  
)
```

= Display if `name` is present

```
#thm-display(  
  thm => thm.name != none  
)
```

4 Display only theorems/ proofs

Theorem 1.1 (Name). *Lorem ipsum dolor sit amet, consectetur adipiscing.*

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing. ■

5 Display if name is present

Theorem 1.1 (Name). *Lorem ipsum dolor sit amet, consectetur adipiscing.*

Definition 2.1 (Thing). Lorem ipsum.

fmt **function** or **auto**

Formatting function, with the same form as `thm.fmt`. The default **auto** uses the originally supplied formatting function `thm.fmt`.

```
#thm-display(  
  thm => thm.supplement != "Proof",  
  final: true,  
  fmt: thm => {  
    let head = [*#thm.supplement~#thm.number*]  
    if thm.name != none {  
      head = head + [~(#thm.name)]  
    }  
    let page = link(thm.loc,  
    [#thm.loc.position().page])  
    [#head~#box(width: 1fr, repeat[.])~#page\ ]  
  }  
)
```

Theorem 1.1 (Name)	17
Lemma 2.1	17
Definition 2.1 (Thing)	17
Lemma 2.2	17

Setting `final: true` ensures that even if this `thm-display()` call is placed at the beginning of the document, all theorem environments are listed.

Default: **auto**

at **label** or **selector** or **location** or **function** or **auto**

Location up to which theorem environments will be displayed. The default **auto** uses the location where `thm-display()` was called.

= Display up to <h2>

```
#thm-display(at: <h2>)
```

6 Display up to <h2>

Theorem 1.1 (Name). *Lorem ipsum dolor sit amet, consectetur adipiscing.*

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing. ■

Default: `auto`

final `bool`

If `true`, display all theorem environments up to the end of the document. Useful for creating lists of theorems in the beginning of documents, before they've been stated. Overrides `thm-display.at`.

Default: `false`

thm-stored `state`

State containing theorem environment data, as an array of `thm` dictionaries.

A.3. `thm-counter`

Exposes methods for manipulating counters used by theorem environments. Import all functions using

```
#import thm-counter: *
```

- `thm-counter-get()`
- `thm-counter-step()`
- `thm-counter-update()`

Variables:

- `thm-counters`

`thm-counter-get`

Get theorem counter. Must be wrapped in context. See `thm.counter`.

Parameters

`thm-counter-get(counter: str) -> array`

`thm-counter-step`

Step theorem counter forward, relative to a `base` and `base-level`. Must be wrapped in context. See `thm.counter`, `thm.base`, `thm.base-level`.

Parameters

```
thm-counter-step(  
    counter: str,  
    base: str,  
    base-level: int  
)
```

`thm-counter-update`

Update theorem counter See `thm.counter`.

Parameters

```
thm-counter-update(  
    counter: str,
```

```
    update: function  
  )
```

thm-counters **state**

State containing a dictionary of theorem environment counters. Each dictionary key is a counter name, and each entry is a counter value (array of integers).

B. Themes

B.1. thm-themes.ams

This theme defines the following versions of `thm` in the AMS style.

- `thm` in the plain style, intended for theorems, lemmas, corollaries, propositions, conjectures.
- `thm-def` in the definition style, intended for definitions, conditions, problems, examples.
- `thm-rem` in the remark style, intended for remarks, notes, annotations, claims, cases, acknowledgments, conclusions.

These styles have been used to provide the following environments.

- `theorem`, `proposition`, `lemma`, `conjecture`, `definition` sharing the counter "Theorem", with the default base "heading".
- `corollary`, `example` sharing the counter "Sub-Theorem", with base "Theorem".
- `problem` with its own counter "Problem", with the default base.
- `remark`, `claim`, `proof` (un-numbered).

```
#import "@preview/ctheorems:2.0.0": *
#import thm-themes.ams: *
#show: thm-rules

#set heading(numbering: "1.")

= Convergence in Probability

#definition[
  We say that  $\{X_n\}$  converges to  $X$  in
  probability if for every  $\epsilon > 0$ ,
   $\mathbb{P}(|X_n - X| > \epsilon) \rightarrow 0$ 
  as  $n \rightarrow \infty$ .
]
#remark[
  This is denoted  $X_n \xrightarrow{p} X$ .
]
#example[
  Let  $\mathbb{E}[X_n] = 0$  and  $\mathbb{E}[X_n^2] \rightarrow 0$ .
  Then,  $X_n \xrightarrow{p} 0$ .
]
#theorem[Weak Law of Large Numbers][
  Let  $\{X_n\}$  be i.i.d. with  $\mathbb{E}[X_1] = \mu$ ,
   $\mathbb{E}[X_1^2] < \infty$ . Then
   $\frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{p} \mu$ .
]
#proof[
  #lorem(14)
]
```

1. Convergence in Probability

Definition 1.1. We say that $\{X_n\}$ converges to X in probability if for every $\epsilon > 0$,

$$\mathbb{P}(|X_n - X| > \epsilon) \rightarrow 0$$

as $n \rightarrow \infty$.

Remark. This is denoted $X_n \xrightarrow{p} X$.

Example 1.1.1. Let $\mathbb{E}[X_n] = 0$ and $\mathbb{E}[X_n^2] \rightarrow 0$. Then, $X_n \xrightarrow{p} 0$.

Theorem 1.2 (Weak Law of Large Numbers).
Let $\{X_n\}$ be i.i.d. with $\mathbb{E}[X_1] = \mu$, $\mathbb{E}[X_1^2] < \infty$.
Then

$$\frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{p} \mu.$$

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut. ■

B.2. thm-themes.stripe

This theme creates theorem styles and environments just like `thm-themes.ams`, but with a formatting function `thm-fmt-stripe` which produces colorful stripes on the left sides of theorems.

```
#import "@preview/ctheorems:2.0.0": *
#import thm-themes.stripe: *
#show: thm-rules

#set heading(numbering: "1.")

= Convergence in Probability

#definition[
  We say that  $\{X_n\}$  converges to  $X$  in
  probability if for every  $\epsilon > 0$ ,
   $\mathbb{P}(|X_n - X| > \epsilon) \rightarrow 0$ 
  as  $n \rightarrow \infty$ .
]
#remark[
  This is denoted  $X_n \xrightarrow{p} X$ .
]
#example[
  Let  $\mathbb{E}[X_n] = 0$  and  $\mathbb{E}[X_n^2] \rightarrow 0$ .
  Then,  $X_n \xrightarrow{p} 0$ .
]
#theorem[Weak Law of Large Numbers][
  Let  $\{X_n\}$  be i.i.d. with  $\mathbb{E}[X_1] = \mu$ ,
   $\mathbb{E}[X_1^2] < \infty$ . Then
   $\frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{p} \mu$ .
]
#proof[
  #lorem(14)
]
```

1. Convergence in Probability

Definition 1.3. We say that $\{X_n\}$ converges to X in probability if for every $\epsilon > 0$,

$$\mathbb{P}(|X_n - X| > \epsilon) \rightarrow 0$$

as $n \rightarrow \infty$.

Remark. This is denoted $X_n \xrightarrow{p} X$.

Example 1.3.1. Let $\mathbb{E}[X_n] = 0$ and $\mathbb{E}[X_n^2] \rightarrow 0$. Then, $X_n \xrightarrow{p} 0$.

Theorem 1.4 (Weak Law of Large Numbers). Let $\{X_n\}$ be i.i.d. with $\mathbb{E}[X_1] = \mu$, $\mathbb{E}[X_1^2] < \infty$. Then

$$\frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{p} \mu.$$

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut. ■

thm-fmt-stripe

Theorem formatting function, producing a colorful stripe on the left side. Wrapper around `thm-fmt-block()`.

Parameters

```
thm-fmt-stripe(
  thm: dictionary,
  ..args: arguments,
  stripe: stroke or none,
  inset: relative dictionary
) -> content
```

..args arguments

Arguments, passed on to `thm-fmt-block()`

stripe stroke or none

Stroke of the stripe.

Default: 2pt + black

inset relative or dictionary

Inset for the block.

Default: (x: 1.0em, y: 0.2em)